Syncra API Contract

Status: Draft v0.2 — MVP scope locked **Demo target:** June 16, 2026 **Owners:** Backend (A, B, C) + Frontend **Last updated:** May 12, 2026

Single source of truth for every endpoint. **Update this file BEFORE writing code.** If frontend and backend disagree on a shape, this file wins.

---

0. Locked decisions (don't re-litigate)

Decision	Choice
Backend stack	Python 3.12 + FastAPI
Database	Firestore (Firebase)
File storage	Firebase Cloud Storage
Auth	Firebase Auth (Admin SDK verifies tokens)
Push notifications	Out of scope for v1 (in-app only)
API style	REST (OpenAPI auto-generated by FastAPI)
Long-running tasks	Polling (no SSE/WebSockets for v1)
Authorization model	Owner-only — no resource sharing
Rate limiting	Per-user, soft caps on LLM-heavy endpoints
Resume rendering	LaTeX via Tectonic (slim Docker image)
Job matching	Claude as LLM-judge (no embeddings, no vector DB)
LLM provider	Anthropic (Claude) for reasoning + tailoring + matching
Agent trigger	On-demand when user opens app (not scheduled)
Chatbot	Wired to real reasoner; full response at once (frontend simulates streaming)
Deployment	Railway
Package manager	uv

---

1. Conventions

Base URL

- Dev: `http://localhost:8000/api/v1`
- Prod: `https://syncra-api.up.railway.app/api/v1`

Auth

Every endpoint except `/health` requires:

```
Authorization: Bearer <firebase_id_token>
```

Backend verifies with `firebase-admin`, extracts UID. We do NOT issue our own JWTs.

Authorization

Owner-only. Every query filters by the authenticated UID. Users never see another user's resumes, applications, pipeline cards, or conversations. Global `jobs/` collection is read-only and shared.

Request / response format

- JSON unless noted (file uploads use `multipart/form-data`).
- Timestamps: ISO 8601 UTC strings.
- IDs: strings (Firestore document IDs).
- Field names: `snake_case`.

Pagination

Simple cursor (last document ID):

```
GET /resumes?limit=20&cursor=<last_doc_id>
```

Response: `{ "data": [...], "next_cursor": "doc_abc123" | null }`

Error format

json

```
{
  "error": {
    "type": "validation | auth | not_found | rate_limited | server",
    "message": "Human-readable",
    "status_code": 400
  }
}
```

HTTP	type	When
400	validation	Bad body / params
401	auth	Missing / invalid / expired Firebase token
403	auth	Authenticated but not the owner
404	not_found	Doesn't exist or not owned by user
422	validation	Pydantic validation failure
429	rate_limited	Per-user soft cap hit
500	server	Unhandled

Rate limiting

Per-user, daily caps, counters in `users/{uid}/usage/{YYYY-MM-DD}`:

Endpoint	Daily cap
`POST /resumes/upload`	20
`POST /resumes/{id}/tailor`	20
`POST /agent/brief`	12
`POST /agent/chat`	30

Return `429` when exceeded. Frontend shows a friendly message.

---

2. Shared schemas

`User`

```
json
```

```
{
  "id": "firebase_uid",
  "name": "Daryn",
  "email": "daryn@example.com",
  "role": "UX Designer",
  "avatar_url": "https://...",
  "is_agent_active": true,
  "autonomy_level": "suggest | ask_first | auto_apply",
  "created_at": "2026-05-12T09:42:00Z"
}
```

`Resume`

Mirrors `lib/features/resumes/models/resume_file.dart`.

json

```
{
  "id": "resume_abc123",
  "name": "Daryn_resume.pdf",
  "size": 142000,
  "mime_type": "application/pdf",
  "uploaded_at": "2026-05-07T10:30:00Z",
  "source": "manual | tailored",
  "storage_url": "https://firebasestorage.googleapis.com/...",
  "parent_resume_id": null,
  "tailored_for_job_id": null
}
```

`ResumeJSON` (internal structured representation)

Used by the tailor and the LaTeX template. Not returned directly from endpoints.

json

```
{
  "header": { "name": "...", "email": "...", "phone": "...", "location": "...", "
  "summary": "...",
  "experience": [
    { "company": "...", "role": "...", "start": "...", "end": "...", "location":
  ],
  "education": [{ "school": "...", "degree": "...", "start": "...", "end": "...",
  "skills": ["..."],
  "projects": [{ "name": "...", "description": "...", "bullets": [...], "link": "
}
```

`Job`

`Job`

Global; mirrors `lib/data/models/job.dart`.

```json
{
  "id": "job_abc123",
  "title": "Senior Product Designer",
  "company": "Binance",
  "location": "Remote",
  "salary": "$140K-$170K",
  "description": "Full JD text...",
  "source": "jsearch",
  "source_url": "https://...",
  "discovered_at": "2026-05-12T08:00:00Z"
}
```

`PipelineCard`

Per-user, per-job agent decision. Powers the dashboard "Approval Pipeline" + jobs page.

```json
{
  "id": "card_abc123",
  "user_id": "...",
  "job": { /* Job, denormalized snapshot */ },
  "category": "ready | input_needed | exploration",
  "match_score": 94,
  "agent_action": "Proactive Intercept",
  "agent_justification": "Binance posted this 15 mins ago...",
  "matched_skills": ["UX Design", "Figma"],
  "missing_skills": [],
  "why": "Your experience matches...",
  "tailored_resume_id": null,
  "created_at": "2026-05-12T08:01:00Z",
  "status": "pending | approved | dismissed"
}
```

`Application`

Created when user approves a PipelineCard.

json

```
{
  "id": "app_abc123",
  "job": { /* Job, denormalized */ },
  "resume_id": "resume_abc123",
  "pipeline_card_id": "card_abc123",
  "status": "submitted | viewed | replied | interview | offer | rejected",
  "submitted_at": "2026-05-09T14:30:00Z",
  "last_update_at": "2026-05-12T09:42:00Z",
  "next_step": "Behavioral round Thursday 2pm",
  "cover_letter": "..."
}
```

`ChatMessage`

json

```
{
  "id": "msg_abc123",
  "sender": "user | agent",
  "text": "...",
  "attachments": [{ "id": "resume_abc123", "name": "Daryn_resume.pdf" }],
  "tool_calls": [{ "tool": "search_jobs", "args": {}, "result_summary": "Found 12
  "result_cards": [],
  "created_at": "..."
}
```

---

3. Endpoints

3.1 Health — Person C

`GET /health` — no auth, returns `{ "status": "ok", "version": "0.1.0" }`.

3.2 Auth & users — Person C

- `POST /auth/session` — idempotent; creates Firestore user doc on first call. Returns `{ user }`.
- `GET /users/me` → `{ user }`
- `PATCH /users/me` — body: any subset of `{ name, role, autonomy_level, is_agent_active }`
- `DELETE /users/me` → 204 (cascade-deletes subcollections)

3.3 Resumes — Person A

- `GET /resumes` (manual uploads) — query `?limit&cursor` → `{ data, next_cursor }`
- `GET /resumes/tailored` — same shape, source=tailored
- `GET /resumes/{id}` → `{ resume }`
- `POST /resumes/upload` (multipart, field `file`, max 5MB) → 201 `{ resume }`. Side effect: parses to `ResumeJSON`.
- `DELETE /resumes/{id}` → 204
- `POST /resumes/{id}/tailor` body `{ job_id }` → 202 `{ tailoring_job_id, status: "pending", poll_url }`
- `GET /resumes/tailoring/{tj_id}` → `{ status, resume?, error? }`

3.4 Jobs — Person B

- `GET /jobs/{id}` → `{ job }`
- `POST /jobs/search` body `{ query, location?, limit }` → `{ data: [Job] }`. Side effect: upserts to global `jobs/`.

3.5 Agent — Person B (THE HEART)

- `POST /agent/brief` body `{ force?: bool }` → 202 `{ brief_id, status: "pending", poll_url }`
  - Logic: fetch user's active resume → fetch new jobs from JSearch → for each, Claude judges match → write PipelineCards.
  - Frontend checks `users/me.last_brief_at`; if <2h old, just reads `/agent/pipeline`. Else calls this.
- `GET /agent/brief/{brief_id}` → `{ status, progress: { current, total }, card_count?, error? }`
- `GET /agent/pipeline?category&status&limit&cursor` → `{ data: [PipelineCard], next_cursor }`
- `POST /agent/pipeline/{card_id}/approve` body `{ resume_id, auto_submit }` → `{ application }`. Triggers tailoring if not done.
- `POST /agent/pipeline/{card_id}/dismiss` → 204
- `POST /agent/chat` body `{ prompt, attached_resume_ids: [] }` → `{ message: ChatMessage, new_pipeline_cards: [id] }`. Writes any matches to the same pipeline.

3.6 Applications (tracker) — Person C

- `GET /applications?status&limit&cursor` → `{ data, next_cursor }`
- `GET /applications/{id}` → `{ application }`
- `PATCH /applications/{id}` body `{ status?, next_step? }` → `{ application }`

---

4. Firestore data model

```
users/{uid}                              ← User
  ├── resumes/{resume_id}                ← Resume metadata
  │     └── parsed (single doc)          ← ResumeJSON
  ├── pipeline/{card_id}                 ← PipelineCard
  ├── applications/{app_id}              ← Application
  ├── conversations/{conv_id}
  │     └── messages/{msg_id}            ← ChatMessage
  ├── tailoring_jobs/{tj_id}             ← async job status
  ├── briefs/{brief_id}                  ← async brief status
  └── usage/{YYYY-MM-DD}                 ← rate limit counters

jobs/{job_id}                            ← global Job (deduped across users)
```

All user-scoped queries filter by UID. Firestore Security Rules enforce this at DB level too, as defense in depth.

---

5. LLM usage map

Track here so Person A and B don't duplicate prompts. Budgeting cost.

Function	Model	Approx tokens	Used by
Resume parsing (PDF text → ResumeJSON)	`claude-haiku-4-5`	2k in / 1.5k out	upload
Job matching (resume + job → score, gaps)	`claude-haiku-4-5`	3k in / 0.5k out	brief, chat
Resume tailoring (resume + job → tailored ResumeJSON)	`claude-opus-4-7` or `claude-sonnet-4-6`	4k in / 3k out	tailor, approve
Agent reasoning (chat orchestration + tool selection)	`claude-opus-4-7`	varies	chat

Use Haiku for matching (called 20+ times per brief). Reserve Opus/Sonnet for the actual rewrite.

---

6. Resolved open questions

- ✅ Authorization: owner-only, no sharing